

Ex.no:3	DATA CLEANING

Aim:

Algorithm:

Check Missing Values:

1. Start.
2. Import the pandas library.
3. Load the dataset into a DataFrame (df).
4. Display dataset information using df.info().
5. Check missing values using df.isnull().
6. Count missing values using df.isnull().sum().
7. Check missing values using df.isna().
8. Check non-missing values using df.notnull().
9. Display the results.

Drop Missing Values

1. Start.
2. Import the pandas library.
3. Load the dataset into a DataFrame (df).
4. Drop missing rows using dropna().
5. Drop rows (axis=0) or columns (axis=1).
6. Use how='any' to remove rows/columns containing any missing value.
7. Use how='all' to remove rows/columns only if all values are missing.
8. Use thresh=N to keep rows/columns with at least N non-missing values.
9. Use subset=['column_name'] to drop rows based on specific columns.
10. Use inplace=True to modify the original DataFrame.
11. Display the cleaned DataFrame.

Program:

```
import pandas as pd
import os
import kagglehub

# Download latest version
path = kagglehub.dataset_download("nehalbirla/vehicle-dataset-from-cardekho")

print("Path to dataset files:", path)

# Load the vehicle dataset already downloaded using kagglehub
csv_file = [f for f in os.listdir(path) if f.endswith(".csv")][0]

df = pd.read_csv(
    os.path.join(path, csv_file),
    encoding="latin1",
    low_memory=False
)

# Clean column names
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")

# Display selected vehicle dataset columns
data = {
    "Year": df["year"].head(5).tolist(),
    "Car_Name": df["car_name"].head(5).tolist(),
    "Selling_Price": df["selling_price"].head(5).tolist(),
    "Fuel_Type": df["fuel_type"].head(5).tolist()
}

vehicle_df = pd.DataFrame(data)

print("Original DataFrame:")
print(vehicle_df)

print("\n1. First 5 rows (head):")
print(vehicle_df.head())

print("\n2. Last 2 rows (tail):")
print(vehicle_df.tail(2))
```

```

print("\n3. Shape (rows, columns):")
print(vehicle_df.shape)
print("\n4. Column Names:")
print(vehicle_df.columns)
print("\n5. DataFrame Information:")
vehicle_df.info()
print("\n6. Statistical Summary:")
print(vehicle_df.describe())
print("\n7. Vehicles after 2010:")
print(
    df[df["year"] > 2010][
        ["year", "car_name", "selling_price", "fuel_type"]
    ].head()
)

```

Output:

Path to dataset files: /kaggle/input/vehicle-dataset-from-cardekho

Original DataFrame:

	Year	Car_Name	Selling_Price	Fuel_Type
0	2014	ritz	3.35	Petrol
1	2013	sx4	4.75	Diesel
2	2017	ciaz	7.25	Petrol
3	2011	wagon r	2.85	Petrol
4	2014	swift	4.60	Diesel

1. First 5 rows (head):

	Year	Car_Name	Selling_Price	Fuel_Type
0	2014	ritz	3.35	Petrol
1	2013	sx4	4.75	Diesel
2	2017	ciaz	7.25	Petrol
3	2011	wagon r	2.85	Petrol
4	2014	swift	4.60	Diesel

2. Last 2 rows (tail):

	Year	Car_Name	Selling_Price	Fuel_Type
3	2011	wagon r	2.85	Petrol
4	2014	swift	4.60	Diesel

3. Shape (rows, columns):

(5, 4)

4. Column Names:

Index(['Year', 'Car_Name', 'Selling_Price', 'Fuel_Type'], dtype='object')

5. DataFrame Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5 entries, 0 to 4

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	Year	5 non-null	int64
1	Car_Name	5 non-null	object
2	Selling_Price	5 non-null	float64
3	Fuel_Type	5 non-null	object

dtypes: float64(1), int64(1), object(2)

memory usage: 292.0+ bytes

6. Statistical Summary:

	Year	Selling_Price
count	5.000000	5.000000
mean	2013.800000	4.560000
std	2.167948	1.707484
min	2011.000000	2.850000
25%	2013.000000	3.350000
50%	2014.000000	4.600000

75% 2014.000000 4.750000

max 2017.000000 7.250000

7. Vehicles after 2010:

	year	car_name	selling_price	fuel_type
0	2014	ritz	3.35	Petrol
1	2013	sx4	4.75	Diesel
2	2017	ciaz	7.25	Petrol
3	2011	wagon r	2.85	Petrol
4	2014	swift	4.60	Diesel

Result: